



LexIA Maroc

Plateforme SaaS de veille juridique multi-RAG
pour le droit marocain

Documentation technique du projet

Architecture, étapes de réalisation et explication détaillée des algorithmes

Auteur	N'Guémawen N'DJINILA
Cadre	Stage de fin d'année — Transformation Digitale Industrielle
Organisme	Zenithsoft, Rabat
Version applicative	LexIA Maroc v2.0
Corpus indexé	6 353 passages juridiques (travail, fiscal, sociétés, données personnelles)

Résumé

LexIA Maroc est un assistant juridique pour le droit marocain fondé sur une architecture *multi-RAG* (*Retrieval-Augmented Generation*). Le système ne répond jamais « de mémoire » : il récupère, dans un corpus de textes officiels indexés, les passages les plus pertinents, puis un modèle de langage rédige une réponse **exclusivement** à partir de ces passages, en citant l'article, le document et la page exacts. Ce document décrit l'architecture en couches, le pipeline d'ingestion, puis détaille — formules à l'appui — chaque algorithme du pipeline de recherche (embeddings et similarité cosinus, BM25, fusion par rangs réciproques, reclassement par cross-encodeur, score de confiance), la génération, les fonctionnalités métier, la sécurité, le modèle d'abonnement et le déploiement.

Table des matières

Résumé	1
1 Contexte et objectifs	3
1.1 Objectifs fonctionnels	3
2 Architecture globale et principes de conception	3
2.1 Pile technique	3
2.1.1 Pourquoi multi-RAG plutôt qu'un LLM seul ?	4
3 Pipeline d'ingestion des textes juridiques	4
3.1 Extraction du texte (avec OCR de secours)	4
3.2 Suivi de la page source	4
3.3 Découpage en chunks	4
3.4 Snapshots et indexation double	4
4 Le cœur du système : le pipeline multi-RAG	4
4.1 Routage par domaine (classification)	5
4.2 Embeddings denses et similarité sémantique	5
4.3 Recherche lexicale BM25	5
4.4 Expansion de requête	6
4.5 Fusion des listes : Reciprocal Rank Fusion (RRF)	6
4.6 Reclassement par cross-encodeur (reranking)	6
4.7 Score de confiance	7
4.8 Filtre de pertinence de la recherche par référence	7
5 Génération de la réponse (LLM)	7
5.1 Fournisseur et cascade de secours	7
5.2 Adaptation au profil (prompt engineering)	7
5.3 Garde anti-hallucination	7
6 Fonctionnalités métier	8
6.1 Calculateurs juridiques (déterministes)	8
6.2 Audit de contrat	8
6.3 Génération de documents et autres outils	8
7 Sécurité et authentification	8
8 Modèle d'abonnement freemium	9
9 Déploiement en production et tests	9
9.1 Conteneurisation	9
9.2 Configuration de production et qualité	9

10 Bilan, limites et perspectives	9
10.0.1 Limites connues	9
10.0.2 Perspectives	9

1. Contexte et objectifs

Le droit marocain est **dispersé** : Bulletin officiel, codes (travail, commerce, impôts), dahirs, lois-cadres, décrets et circulaires. Identifier la disposition applicable à une situation précise, avec sa référence exacte, exige une expertise et un temps considérables. Les grands modèles de langage (LLM) généralistes n’y répondent pas : ils **hallucinent** des articles, ignorent les textes marocains récents et ne fournissent aucune source vérifiable.

LexIA Maroc répond à ce besoin par une architecture **multi-RAG** : le modèle de langage ne répond qu’à partir de passages de textes officiels **réellement récupérés**, en citant systématiquement leurs sources. C’est la différence fondamentale entre un chatbot généraliste et un assistant juridique de confiance.

1.1 Objectifs fonctionnels

- Répondre en langage naturel (français et arabe) sur quatre domaines : droit du travail, fiscal, des sociétés et protection des données personnelles.
- Ancrer chaque réponse sur des sources officielles et les citer précisément (article + page).
- Ne jamais affirmer ce qui n’est pas dans les textes récupérés (garde anti-hallucination).
- Adapter le registre au profil de l’utilisateur (étudiant, particulier, juriste, avocat, entreprise).
- Compléter le chat par des outils métier : calculateurs, audit de contrat, génération de documents, comparaison de versions.

Le projet s’inscrit dans un stage de fin d’année (Zenithsoft, Rabat) et suit un cahier des charges de dix-huit exigences fonctionnelles (BF-01 à BF-18), en visant une qualité produit.

2. Architecture globale et principes de conception

Le système est organisé en **quatre couches applicatives** plus une couche transverse (sécurité, persistance), avec une **règle de dépendance stricte** : les couches métier dépendent d’un noyau neutre, jamais l’inverse.

`core ← ingestion / processing / retrieval / generation ← api`

Le noyau « core » ne dépend de personne ; la couche « api » orchestre l’ensemble.

Couche	Responsabilité
core	Fondations neutres : registre des domaines, configuration, modèles de base, sécurité, offres.
ingestion	Acquisition : scrapers, extraction PDF/OCR, pipeline d’indexation, veille planifiée.
processing	Transformation : découpage en chunks, classification, calculateurs déterministes.
retrieval	Recherche : embeddings, BM25, expansion, fusion RRF, reranking, routage, confiance.
generation	Réponse : construction du prompt, client LLM avec cascade de secours.
api	Exposition HTTP (FastAPI) : authentification, chat en flux, recherche, outils, abonnement.

2.1 Pile technique

- **Backend** : Python 3.11, FastAPI, SQLAlchemy, base vectorielle Chroma (une collection par domaine), SQLite/PostgreSQL.

- **Modèles ML** : sentence-transformers (embeddings + cross-encodeur), rank-bm25, Tesseract (OCR fr+ar).
- **Génération** : Groq (llama-3.3-70b-versatile), avec OpenAI / Gemini / OpenRouter en secours.
- **Frontend** : React 18, servi en production par nginx.
- **Industrialisation** : Docker Compose, JWT + bcrypt, tests pytest.

Pourquoi multi-RAG plutôt qu'un LLM seul ?

Un LLM seul est une mémoire figée et faillible. Le RAG sépare la **connaissance** (le corpus officiel, à jour, vérifiable) de la **capacité de rédaction** (le LLM). Le modèle ne fait que reformuler des extraits qu'on lui fournit — d'où l'ancrage, la citation des sources et la maîtrise des hallucinations.

3. Pipeline d'ingestion des textes juridiques

Avant toute recherche, les PDF officiels sont transformés en unités indexables, en cinq étapes.

3.1 Extraction du texte (avec OCR de secours)

L'extraction lit d'abord le texte natif via **PyMuPDF** (rapide, fidèle). Pour un PDF **scanné** (image sans couche texte), on bascule sur l'**OCR Tesseract** configuré français *et* arabe (**fra+ara**). Un garde-fou limite le nombre de pages soumises à l'OCR (protection DoS), mais ne s'applique qu'à ce chemin coûteux : un PDF à texte natif de 700 pages (le Code Général des Impôts) est extrait intégralement.

3.2 Suivi de la page source

L'extracteur conserve les **offsets de page**. Après découpage, chaque chunk est relié à sa page d'origine par recherche de sous-chaîne. C'est ce qui permet la citation « *Code du travail 65-99, p. 34* », différenciateur fort face aux concurrents.

3.3 Découpage en chunks

Un texte juridique est structuré en articles. Le chunker exploite cette structure via des motifs (regex) propres à la rédaction marocaine (« Article N », « Art. N », « N — », « Chapitre », « Titre ») pour découper **par article**. À défaut, il retombe sur un découpage par taille : cible de **800 caractères**, chevauchement de **150 caractères**. Le chevauchement évite qu'une information à cheval sur deux chunks soit perdue.

3.4 Snapshots et indexation double

À chaque ingestion, une copie du texte intégral est conservée avec son empreinte (hash), pour la **comparaison de versions**. Chaque chunk est ensuite indexé **deux fois** : dans l'index **vectoriel** (Chroma) pour la recherche sémantique, et dans l'index **lexical** (BM25) pour la correspondance exacte des termes.

4. Le cœur du système : le pipeline multi-RAG

Une question suit une chaîne de traitements dont chaque maillon a un rôle précis.

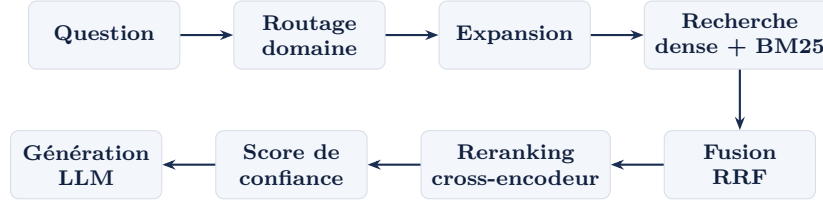


FIGURE 1 – Chaîne de traitement d’une question, du routage à la génération.

4.1 Routage par domaine (classification)

Inutile d’interroger les quatre corpus si la question porte clairement sur le droit du travail. Le routeur classe la question par **comptage de mots-clés**. Point subtil : la correspondance se fait à **limite de mot** (`\b` en regex). Sans cela, un mot-clé court comme « IS » (impôt sur les sociétés) ou « IR » (impôt sur le revenu) serait trouvé par sous-chaîne dans « préavis » ou « secrétaire » — provoquant un mauvais routage.

Soit s_j le nombre de mots-clés du domaine j présents dans la question. La confiance du routage est la part du domaine dominant :

$$\text{conf} = \frac{\max_j s_j}{\sum_j s_j}. \quad (1)$$

Si $\text{conf} \geq 0,3$, on cible le domaine dominant (et ses domaines complémentaires) ; sinon, par prudence, on interroge **tous** les corpus prioritaires.

4.2 Embeddings denses et similarité sémantique

La recherche « dense » projette chaque texte dans un espace vectoriel où la **proximité géométrique traduit la proximité de sens**. Deux formulations d’une même idée (« licenciement abusif » et « rupture injustifiée du contrat ») produisent des vecteurs proches, là où une recherche par mots-clés échouerait.

Le modèle est **intfloat/multilingual-e5-large** (dimension 1024, multilingue fr/ar). Il impose des **préfixes** distincts pour requêtes et passages :

```

QUERY_PREFIX = "query: "      # préfixe des questions
DOC_PREFIX    = "passage: "    # préfixe des passages indexés
emb = model.encode(prefix + texte, normalize_embeddings=True)

```

La proximité entre une requête $\mathbf{q}$ et un document $\mathbf{d}$ est la **similarité cosinus** :

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|} = \frac{\sum_{i=1}^n q_i d_i}{\sqrt{\sum_i q_i^2} \sqrt{\sum_i d_i^2}}. \quad (2)$$

Comme les vecteurs sont **normalisés** à la génération ($\|\mathbf{q}\| = \|\mathbf{d}\| = 1$), la similarité se réduit au **produit scalaire** $\text{sim}(\mathbf{q}, \mathbf{d}) = \mathbf{q} \cdot \mathbf{d}$, ce qui accélère la recherche. Chroma renvoie une distance ; on la convertit par $\text{score} = 1 - \text{distance}$.

4.3 Recherche lexicale BM25

La recherche dense peut « lisser » les termes précis ; or « article 62 » ou une référence exacte doivent être trouvés **à la lettre**. C’est le rôle de **BM25** (*Best Matching 25*), modèle probabiliste de pertinence lexicale. Pour une requête $Q = \{q_1, \dots, q_n\}$ et un document D :

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) (k_1 + 1)}{f(q_i, D) + k_1 \left(1 - b + b \frac{|D|}{\text{avgdl}} \right)}, \quad (3)$$

où $f(q_i, D)$ est la fréquence du terme q_i dans D , $|D|$ la longueur de D et avgdl la longueur moyenne des documents. Le poids **IDF** favorise les termes rares :

$$\text{IDF}(q_i) = \ln \left(\frac{N - n(q_i) + 0,5}{n(q_i) + 0,5} + 1 \right), \quad (4)$$

avec N le nombre total de documents et $n(q_i)$ le nombre de documents contenant q_i . Le paramètre k_1 règle la **saturation** de la fréquence (au-delà d'un seuil, un terme répété n'apporte plus grand-chose) et $b \in [0, 1]$ la **normalisation par la longueur** (pénalise les documents longs). BM25 apporte la précision lexicale que le dense n'offre pas.

4.4 Expansion de requête

Avant de chercher, le système génère quelques **variantes** de la question (reformulations, synonymes juridiques). Chaque variante produit sa propre liste, ce qui augmente le **rappel** : on rate moins de passages pertinents formulés autrement que la question d'origine.

4.5 Fusion des listes : Reciprocal Rank Fusion (RRF)

On dispose de plusieurs listes classées (dense et BM25, pour chaque variante). Leurs scores ne sont **pas à la même échelle** : une similarité cosinus dans $[0, 1]$ contre un score BM25 non borné. **RRF** contourne le problème en ne combinant que les **rangs** :

$$\text{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \text{rang}_r(d)}, \quad k = 60, \quad (5)$$

où $\mathcal{R}$ est l'ensemble des listes et $\text{rang}_r(d)$ la position de d dans la liste r ($0 = \text{tête}$). Un document bien classé dans **plusieurs** listes cumule les contributions et remonte. Le terme $+k$ atténue l'écart entre les toutes premières positions, rendant la fusion robuste au bruit. RRF est sans paramètre à entraîner et très efficace pour fusionner des signaux hétérogènes — d'où son choix.

4.6 Reclassement par cross-encodeur (reranking)

Les embeddings ci-dessus proviennent d'un **bi-encodeur** : **q** et **d** sont encodés **séparément**, puis comparés. Rapide (passages pré-encodés) mais approximatif, car l'encodage d'un passage ignore la requête. On affine avec un **cross-encodeur** (**ms-marco-MiniLM**) appliqué aux seuls meilleurs candidats.

Bi-encodeur	enc(q) et enc(d) indépendants, puis cosinus. Rapide : premier tri sur tout le corpus.
Cross-encodeur	Traite la paire (q, d) ensemble dans un même passage avant du modèle. Bien plus fin, mais coûteux : réservé au re-tri des ~ 10 meilleurs.

Le cross-encodeur renvoie un **logit** s non borné, ramené dans $[0, 1]$ par une **sigmoïde** :

$$\sigma(s) = \frac{1}{1 + e^{-s}}. \quad (6)$$

Un **bonus de correspondance d'article** complète le modèle : si la question cite « article 52 » et que le passage provient de l'article 52, son score est multiplié par 1,5 (plafonné à 1) — le cross-encodeur seul ne privilégie pas toujours la correspondance exacte.

4.7 Score de confiance

Le score de confiance affiché est celui du **meilleur** passage retenu (déjà normalisé). Il est traduit en label qualitatif, jamais en pourcentage trompeur :

$$\text{label}(c) = \begin{cases} \text{élevé} & c \geq 0,80, \\ \text{moyen} & 0,60 \leq c < 0,80, \\ \text{faible} & 0,40 \leq c < 0,60, \\ \text{insuffisant} & c < 0,40. \end{cases} \quad (7)$$

En dessous de 0,40, le système renonce et affiche « aucun texte pertinent trouvé » plutôt qu’une réponse hasardeuse.

4.8 Filtre de pertinence de la recherche par référence

Lorsqu’aucun mot-clé ne correspond, le repli vectoriel renvoie **toujours** les voisins les plus proches — même hors sujet, car sur un corpus juridique tout se ressemble ($\sim 0,8$ de similarité). La requête « note circulaire TVA » (aucune circulaire indexée) remontait ainsi cinq passages sans rapport. La correction impose un **recouvrement lexical** : un résultat n’est retenu que s’il contient réellement au moins un mot significatif t de la requête,

$$\exists t \in \text{tokens}(Q), \quad t \subseteq D,$$

sinon la liste est vide et l’interface l’annonce honnêtement.

5. Génération de la réponse (LLM)

Les passages retenus sont assemblés dans un **prompt** soumis au modèle, qui ne fait que **rédigé** à partir de ce contexte.

5.1 Fournisseur et cascade de secours

Le fournisseur par défaut est **Groq** (llama-3.3-70b-versatile) : gratuit et rapide ($\sim 7-8$ s). Une **cascade** bascule sur le modèle suivant en cas d’erreur **recupérable** :

401 (clé invalide)	Fatal — changer de modèle n’y changera rien, on arrête.
402, 408, 429, 500–504	Récupérable — crédits/limite/timeout/panne : on tente le suivant.
Coupure réseau / DNS	Récupérable — incident possiblement transitoire ou localisé.

En mode **streaming** (affichage progressif via SSE), la bascule n’est possible qu’avant le premier token émis : après, rebasculer casserait l’affichage client.

5.2 Adaptation au profil (prompt engineering)

Le prompt système est composé selon le rôle : cinq **personas**, chacun avec un ton et une structure propres.

5.3 Garde anti-hallucination

Des règles strictes encadrent la génération : service **exclusivement** marocain (ne jamais présenter un texte comme étranger — incident réel : la loi 31-13 qualifiée « de loi française ») ; aucune affirmation hors des sources fournies ; interdiction des réponses « cul-de-sac » (indiquer ce que couvrent les corpus et inviter à reformuler plutôt qu’un « aucune information » sec). Un **budget de contexte** (comptage de tokens via tiktoken) élague les passages excédentaires pour ne jamais dépasser la fenêtre du modèle.

Étudiant	Pédagogique, définitions. Sections « Analyse » / « Ce qu'il faut retenir ».
Particulier	Vulgarisé, accessible, orienté action.
Juriste / Avocat	Technique, références d'articles exactes, argumentaire.
Entreprise	Orienté dirigeant. Sections « Obligations » / « Risques » / « Actions ».

6. Fonctionnalités métier

6.1 Calculateurs juridiques (déterministes)

Contrairement au chat, ces calculs **n'utilisent pas de LLM** : déterministes, reproductibles, fondés sur le barème légal avec référence à l'article. L'**indemnité de licenciement** (article 53) suit un barème progressif en heures de salaire par année d'ancienneté. Soit w le salaire mensuel et a l'ancienneté ; le taux horaire est $\tau = w/191$ et le nombre d'heures :

$$H(a) = \sum_t h_t \cdot \min(\max(a - a_{t-1}, 0), a_t - a_{t-1}), \quad (8)$$

avec les tranches (h_t) de l'article 53 : 96 h/an (années 1–5), 144 h/an (6–10), 192 h/an (11–15), 240 h/an (au-delà). L'indemnité vaut $\tau \cdot H(a)$. S'ajoutent le **préavis** (article 51) et le **salaire net** (CNSS 4,48% plafonnée, AMO 2,26%, IR au barème progressif).

6.2 Audit de contrat

L'utilisateur importe un contrat ; le système extrait son texte, **récupère les articles de loi pertinents**, puis génère un rapport structuré (conformité, risques, clauses manquantes). Point clé : l'audit interroge le corpus **matière par matière** (période d'essai, préavis, salaire, congés, rupture, durée du travail) plutôt qu'en une requête unique — sans quoi des articles décisifs (comme l'**article 14** sur la durée maximale de la période d'essai) n'étaient jamais récupérés, et l'audit concluait à tort à la conformité.

6.3 Génération de documents et autres outils

Quatre modèles déterministes conformes au Code du travail — **attestation de travail** (art. 72), **mise en demeure, contrat CDI** (art. 14, 51, 53), **reçu pour solde de tout compte** (art. 73–74) — remplis par formulaire et exportables en **Word** et **PDF**. S'y ajoutent la **comparaison de versions** (diff sur snapshots), la **recherche par référence**, la **veille réglementaire** et le **support de l'arabe** (RTL).

7. Sécurité et authentification

- **JWT + bcrypt** : mots de passe hachés (bcrypt, 12 rounds) ; jeton d'accès de courte durée gardé **en mémoire** côté client (jamais dans `localStorage` — résistance au XSS).
- **Refresh token en cookie httpOnly** : invisible au JavaScript ; en production, drapeau **Secure** (HTTPS obligatoire).
- **Rotation du refresh token** : chaque rafraîchissement révoque l'ancien et en émet un nouveau. Un correctif « single-flight » mutualise les rafraîchissements concurrents (React StrictMode en lançait deux, dont le second échouait et déconnectait l'utilisateur).
- **Rate limiting** : 60 requêtes / 60 s par IP.
- **RBAC** et verrouillage de fonctionnalités par offre ; en-têtes de sécurité (CSP, HSTS, X-Frame-Options) appliqués par nginx.

8. Modèle d'abonnement freemium

La mécanique d'abonnement — offres, quotas, compteur d'usage, paywall — est complète et prête à être reliée à un prestataire (**Stripe** ou **CMI** au Maroc) via un point d'activation unique aujourd'hui « simulé ».

Offre	Prix	Contenu
Gratuit	0 MAD	20 questions/mois. Chat sourcé, calculateurs, recherche par référence, comparaison.
Pro	99 MAD	Questions illimitées + audit de contrat + génération de documents + export (JSON/Word).

Chaque fonctionnalité à forte valeur déclare la clé qui la déverrouille ; l'API refuse (HTTP 402) l'accès si l'offre ne l'inclut pas ou si le quota mensuel est épuisé.

9. Déploiement en production et tests

9.1 Conteneurisation

Trois services orchestrés par Docker Compose : l'**API** (uvicorn, 2 workers, utilisateur non-root, modèles ML préchargés dans l'image pour éviter un téléchargement de 2,2Go au premier démarrage), le **frontend** (build React servi par nginx) et le **watcher** (veille). Des *healthchecks* gouvernent l'ordre de démarrage.

9.2 Configuration de production et qualité

`ENVIRONMENT=production` désactive la documentation d'API et active les cookies Secure ; l'API refuse de démarrer si `SECRET_KEY` n'a pas été changée ; HTTPS terminé par un reverse proxy ; CORS restreint. Le projet est couvert par **105 tests** (unitaires et d'intégration). Plusieurs bugs réels n'ont été détectés qu'en **exécutant les vrais parcours** : recherche d'historique cassée, session perdue au rechargement, recherche par référence hors sujet, audit passant à côté d'une non-conformité — tous corrigés et couverts par des tests de non-régression.

10. Bilan, limites et perspectives

LexIA Maroc dépasse le stade du prototype : architecture propre en couches, pipeline multi-RAG complet (dense + BM25 + RRF + reranking + confiance), réponses sourcées à la page, outils métier, sécurité soignée, abonnement freemium et déploiement conteneurisé documenté.

Limites connues

- **Corpus jurisprudence vide** : les décisions de la Cour de cassation ne sont pas disponibles sous forme d'un recueil PDF exploitable proprement.
- **OCR des scans** : l'installation de Tesseract sur le poste a été bloquée par les droits administrateur ; les dahirs/arrêtés scannés ne sont pas encore océrisés.
- **Paiement non branché** : la mécanique est complète, le prestataire réel reste à connecter.

Perspectives

- Brancher le paiement réel (Stripe ou CMI) sur le point d'activation déjà prévu.
- Ajouter les conditions générales et la politique de confidentialité (conformité loi 09-08).
- Sourcer et indexer un corpus de jurisprudence.

- Observabilité (latence, coût par requête) et détection automatique de la langue.

Document technique du projet LexIA Maroc — Stage de fin d'année, Zenithsoft (Rabat). Les réponses fournies par la plateforme sont informatives et ne constituent pas un avis juridique professionnel.